LABORATORY WORK NO. 3

Polyalphabetic Ciphers

Task 3.2 — the Vigenère cipher
implemented for the Romanian alphabet

Discipline: **Cryptography**

Author: **Mutu Adrian**

Group: **FAF-242**

1 November 2026

1. Short theory

A monoalphabetic cipher replaces every occurrence of a plaintext letter with one and the same ciphertext letter. That single property is what broke the cipher of Laboratory Work No. 2: the substitution is fixed, so the frequency profile of the language passes through into the cryptogram unchanged and the key can be recovered by counting letters, without ever searching the key space.

A *polyalphabetic* cipher removes exactly that property. The substitution applied to a letter is no longer constant — it depends on where the letter stands in the text. If the letter T is sometimes enciphered as `a` and sometimes as `b`, and X is also sometimes enciphered as `a` and sometimes as `b`, then the high frequency of T is blended with the low frequency of X and the resulting distribution is flatter than either. The simplest illustration uses two alphabets alternately, one for the even positions and one for the odd ones, for instance the two permutations

$$p_1(a) = (3 \cdot a) \bmod 26 \quad \text{and} \quad p_2(a) = (7 \cdot a + 13) \bmod 26.$$

The first polyalphabetic system is credited to Leon Battista Alberti, and the method that carries the name of Blaise de Vigenère was in fact published earlier, in 1553, by Giovan Battista Bellaso; Vigenère himself described a stronger, related cipher in 1586. The name stuck to the simpler construction anyway.

The Vigenère cipher

The Vigenère cipher is a Caesar cipher whose shift changes at every position. The key is not one number but a sequence of shifts — in practice a word, so that it can be memorised — which is repeated from the beginning as many times as the text requires. Writing m_i, c_i and k_i for the numeric codes of the i-th letter of the plaintext, of the cryptogram and of the key stream, and n for the size of the alphabet:

$$c_i = (m_i + k_i) \bmod n \quad \text{and} \quad m_i = (c_i - k_i) \bmod n$$

with $k_i = \text{key}[i \bmod |\text{key}|]$. For the Romanian alphabet $n = 31$ and the letters are coded $A = 0$, $Ă = 1$, $Â = 2$, …, $Z = 30$. As in Laboratory Work No. 1, the subtraction is written as $(c_i - k_i + n) \bmod n$, because in several languages the remainder of a negative number is itself negative.

The same transformation can be carried out without arithmetic, using the *Tabula Recta* — the square in which row `k` holds the alphabet rotated by `k` positions. To encipher, one looks up the row of the key letter and the column of the plaintext letter; the cell where they meet is the ciphertext letter. The table is only a different presentation of `(m + k) mod n`, and the program prints it for the 31 Romanian letters.

Why the cipher is stronger — and where it still fails

Two things improve over the monoalphabetic case. The first is the key space. The assignment requires a key of at least 7 letters, and over 31 letters that already gives

$$31^7 = 27\,512\,614\,111 \approx 2.75 \times 10^{10}$$

keys of that length alone — against the 30 keys of the Caesar cipher of Laboratory Work No. 1 — and the attacker does not know the length either, so the lengths above 7 must be searched as well. The twelve-letter key used in the runs below is one of $31^{12} \approx 7.9 \times 10^{17}$.

The second, and the more important one, is that the frequency profile of the language no longer survives. A plaintext letter is enciphered through $|\text{key}|$ different Caesar alphabets depending on its position, so

one plaintext letter produces several different ciphertext letters and one ciphertext letter stands for several different plaintext letters. Section 3 shows both effects happening inside a single 36-letter message.

The cipher is nevertheless breakable, and the attack is classical. Because the key repeats, two identical groups of letters in the plaintext that happen to fall on the same offset of the key produce two identical groups in the cryptogram; the distance between such repetitions is a multiple of the key length. Collecting those distances and taking their common divisors — the Kasiski examination — reveals `|key|`, which the index of coincidence (the Friedman test) estimates statistically as well. Once the length is known the cryptogram is split into `|key|` columns, each of which was enciphered with a single shift, and every column is then broken by the frequency analysis of Laboratory Work No. 2. The cipher is therefore secure only while the key is long and never reused — in the limit, a key as long as the message and used once is the one-time pad, which is provably unbreakable.

2. Implementation

The program is written in Python 3 and uses only the standard library. It is organised like the previous laboratory works: the alphabet and the validation in one module, the cipher in another, and the interaction in a third.

File	Contents
`romanian_alphabet.py`	the 31-letter encoding and all input validation
`vigenere.py`	the key stream, encryption, decryption, the worked table and the Tabula Recta
`main.py`	the interactive program
`tests/test_vigenere.py`	automated checks, including the example worked out by hand in the assignment

The letter encoding

```
ALPHABET = (
    "A", "Ă", "Â", "B", "C", "D", "E", "F", "G", "H", "I", "Î", "J", "K", "L",
    "M", "N", "O", "P", "Q", "R", "S", "Ș", "T", "Ț", "U", "V", "W", "X", "Y",
    "Z",
)

N = len(ALPHABET)  # 31
```

A letter's numeric code is its index in this tuple, which is what the assignment means by "codificate cu numerele 0, 1, … 30". No `ord()` or `chr()` appears anywhere in the cipher, so the diacritics are not a special case: `Ă`, `Â`, `Î`, `Ș` and `Ț` are letters at positions 1, 2, 11, 22 and 24, and the wrap-around from `Z` back to `A` needs no extra handling.

The assignment states the permitted characters as "între 'A' și 'Z', 'a' și 'z'" while also asking for the 31-letter Romanian alphabet; the two are reconciled by reading the range as the Romanian A … Z, diacritics included, which is the alphabet the message is written in. Lowercase input is accepted and uppercased, and `Ş`/`Ţ` typed with a cedilla are folded onto `Ș`/`Ț` with a comma below, since both spellings circulate on keyboards.

Preparing the text

The assignment prescribes the preparation exactly: "în mesaj mai întâi trebuie eliminate spațiile, apoi toate literele se vor transforma în majuscule". One function does both and rejects everything else:

```
def normalize_text(raw):
    text = unicodedata.normalize("NFC", raw)

    letters = []
    for position, char in enumerate(text, start=1):
        if char.isspace():          # spaces are allowed in the input...
            continue                # ...and simply dropped
        char = CEDILLA_VARIANTS.get(char, char)
        char = LOWER_TO_UPPER.get(char, char)
        if char not in LETTER_TO_CODE:
            raise InvalidCharacterError(char, position)
        letters.append(char)
    return "".join(letters)
```

The exception carries the offending character and its position, so the message printed to the user names what was rejected, where it was, and what the permitted range is — the "i se va sugera diapazonul corect al caracterelor" of the assignment.

The key

The key is validated through the same function, with one additional rule — the length:

```
MIN_KEY_LENGTH = 7       # "Lungimea cheii nu trebuie sa fie mai mica de 7"


def parse_key(raw):
    try:
        key = normalize_text(raw)
    except InvalidCharacterError as error:
        raise InvalidKeyError(
            "the key may contain only letters of the Romanian alphabet: %s"
            % error
        )
    if len(key) < MIN_KEY_LENGTH:
        raise InvalidKeyError(
            "the key %r has %d letter(s); it must have at least %d"
            % (key, len(key), MIN_KEY_LENGTH)
        )
    return key
```

Because the key passes through `normalize_text` first, spaces in it are removed before the length is measured, so `c h e i a a` is six letters and is rejected.

Encryption and decryption

The key stream is the key repeated for as long as the text needs it, and the two ciphers are the two formulas of section 1, applied letter by letter:

```
def key_stream(key, length):
    """Repeat `key` until it covers `length` letters: k_i = key[i mod |key|]."""
    return "".join(key[position % len(key)] for position in range(length))


def encrypt(text, key, alphabet=ALPHABET):
    """c_i = (m_i + k_i) mod n, letter by letter."""
    to_code = _codes(alphabet)
    n = len(alphabet)
    stream = key_stream(key, len(text))
    return "".join(
        alphabet[(to_code[letter] + to_code[k]) % n]
        for letter, k in zip(text, stream)
    )


def decrypt(text, key, alphabet=ALPHABET):
    """m_i = (c_i - k_i) mod n, letter by letter."""
    to_code = _codes(alphabet)
    n = len(alphabet)
    stream = key_stream(key, len(text))
    return "".join(
        alphabet[(to_code[letter] - to_code[k] + n) % n]
        for letter, k in zip(text, stream)
    )
```

The ordered alphabet is a parameter rather than a constant. That is what makes it possible to run the very same functions over the 26 letters of English and compare the output against the example printed in the assignment, before trusting the code on 31 letters — the check is in section 3.

Two further functions exist only to make the cipher visible. `steps()` returns the six rows of the table used in the assignment to work the cipher by hand — the input letter, the key letter, their codes, the result of the modular addition and the output letter — and `tabula_recta()` builds the 31 × 31 square described in section 1.

3. Results

All the runs below were produced by `python3 main.py` and are reproduced as they appear on screen.

Encryption

```
======================================================================
Task 3.2 — Vigenere cipher (Romanian alphabet, n = 31)
======================================================================
Operation — 1 = encrypt, 2 = decrypt: 1
Key (at least 7 Romanian letters): criptografie
Plaintext (spaces are allowed and removed): Criptografia este arta de a pastra secrete

  Alphabet in use (n = 31):
    code    0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
    letter  A  Ă  Â  B  C  D  E  F  G  H  I  Î  J  K  L  M
    code   16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
    letter  N  O  P  Q  R  S  Ș  T  Ț  U  V  W  X  Y  Z

  Key used: CRIPTOGRAFIE  (repeated to the length of the text)

  The transformation, letter by letter:
    Plaintext  M   C  R  I  P  T  O  G  R  A  F  I  A  E  S  T  E
    Key        K   C  R  I  P  T  O  G  R  A  F  I  E  C  R  I  P
    M (codes)      4 20 10 18 23 17  8 20  0  7 10  0  6 21 23  6
    K (codes)      4 20 10 18 23 17  8 20  0  7 10  6  4 20 10 18
    M+K (mod 31)   8  9 20  5 15  3 16  9  0 14 20  6 10 10  2 24
    Ciphertext C   G  H  R  D  M  B  N  H  A  L  R  E  I  I  Â  Ț

    Plaintext  M   A  R  T  A  D  E  A  P  A  S  T  R  A  S  E  C
    Key        K   T  O  G  R  A  F  I  E  C  R  I  P  T  O  G  R
    M (codes)      0 20 23  0  5  6  0 18  0 21 23 20  0 21  6  4
    K (codes)     23 17  8 20  0  7 10  6  4 20 10 18 23 17  8 20
    M+K (mod 31)  23  6  0 20  5 13 10 24  4 10  2  7 23  7 14 24
    Ciphertext C   T  E  A  R  D  K  I  Ț  C  I  Â  F  T  F  L  Ț

    Plaintext  M   R  E  T  E
    Key        K   A  F  I  E
    M (codes)     20  6 23  6
    K (codes)      0  7 10  6
    M+K (mod 31)  20 13  2 12
    Ciphertext C   R  K  Â  J

  Plaintext  (uppercase, no spaces) : CRIPTOGRAFIAESTEARTADEAPASTRASECRETE
  Cryptogram                        : GHRDMBNHALREIIÂȚTEARDKIȚCIÂFTFLȚRKÂJ
```

Every column can be checked by hand against the Tabula Recta. The first one is `C = 4` with key `C = 4`, giving `(4 + 4) mod 31 = 8 = G`; the sixteenth is `E = 6` with key `P = 18`, giving `(6 + 18) mod 31 = 24 = Ț` — a plain letter enciphered as a diacritic one, which the natural wrap-around of the 31-letter alphabet produces by itself.

The polyalphabetic effect

This single 36-letter message is already enough to show what the cipher is for. The plaintext letter `A` occurs seven times and, depending on the key letter standing over it, comes out as six different letters; conversely, the ciphertext letter `I` stands for three different plaintext letters in the same message:

Plaintext letter	A	A	A	A	A	A
Key letter	A	E	T	R	I	C
Becomes	A	E	T	R	I	C

Ciphertext letter	I	I	I
Key letter	C	R	I
Stands for	E	S	A

A monoalphabetic cipher can do neither. This is precisely the property that makes the letter-counting attack of Laboratory Work No. 2 useless here: the frequency of `A` in Romanian is spread over six ciphertext letters, and the count of `I` in the cryptogram is a mixture of the counts of three unrelated plaintext letters.

Decryption

The cryptogram produced above is fed back in with the same key:

```
Operation – 1 = encrypt, 2 = decrypt: 2
Key (at least 7 Romanian letters): criptografie
Cryptogram (spaces are allowed and removed): GHRDMBNHALREIIÂȚTEARDKIȚCIÂFTFLȚRKÂJ

  Key used: CRIPTOGRAFIE  (repeated to the length of the text)

  The transformation, letter by letter:
    Ciphertext C   G  H  R  D  M  B  N  H  A  L  R  E  I  I  Â  Ț
    Key        K   C  R  I  P  T  O  G  R  A  F  I  E  C  R  I  P
    C (codes)      8  9 20  5 15  3 16  9  0 14 20  6 10 10  2 24
    K (codes)      4 20 10 18 23 17  8 20  0  7 10  6  4 20 10 18
    C–K (mod 31)   4 20 10 18 23 17  8 20  0  7 10  0  6 21 23  6
    Plaintext  M   C  R  I  P  T  O  G  R  A  F  I  A  E  S  T  E

    Ciphertext C   T  E  A  R  D  K  I  Ț  C  I  Â  F  T  F  L  Ț
    Key        K   T  O  G  R  A  F  I  E  C  R  I  P  T  O  G  R
    C (codes)     23  6  0 20  5 13 10 24  4 10  2  7 23  7 14 24
    K (codes)     23 17  8 20  0  7 10  6  4 20 10 18 23 17  8 20
    C–K (mod 31)   0 20 23  0  5  6  0 18  0 21 23 20  0 21  6  4
    Plaintext  M   A  R  T  A  D  E  A  P  A  S  T  R  A  S  E  C

    Ciphertext C   R  K  Â  J
    Key        K   A  F  I  E
    C (codes)     20 13  2 12
    K (codes)      0  7 10  6
    C–K (mod 31)  20  6 23  6
    Plaintext  M   R  E  T  E

  Cryptogram (uppercase, no spaces) : GHRDMBNHALREIIÂȚTEARDKIȚCIÂFTFLȚRKÂJ
  Decrypted message                 : CRIPTOGRAFIAESTEARTADEAPASTRASECRETE
```

The original message is recovered exactly. As the assignment notes, putting the spaces back — *CRIPTOGRAFIA ESTE ARTA DE A PĂSTRA SECRETE* — is done by the reader, from the sense of the text.

Invalid input

The program does not stop on bad input; it explains the rejection and asks again. The run below rejects a key of 5 letters, then a key containing `!`, then a message containing a digit:

```
Operation - 1 = encrypt, 2 = decrypt: 1
Key (at least 7 Romanian letters): super
  [invalid input] the key 'SUPER' has 5 letter(s); it must have at least 7

Key (at least 7 Romanian letters): cheie!
  [invalid input] the key may contain only letters of the Romanian alphabet:
  the character '!' (position 6) is not a letter of the Romanian alphabet;
  allowed letters are A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W
  X Y Z (uppercase or lowercase) and spaces, which are removed

Key (at least 7 Romanian letters): cheiamea
Plaintext (spaces are allowed and removed): mesaj de proba 7
  [invalid input] the character '7' (position 16) is not a letter of the
  Romanian alphabet; allowed letters are A Ă Â B C D E F G H I Î J K L M N O P
  Q R S Ș T Ț U V W X Y Z (uppercase or lowercase) and spaces, which are
  removed

Plaintext (spaces are allowed and removed): mesaj de proba

  Key used: CHEIAMEA  (repeated to the length of the text)

  The transformation, letter by letter:
    Plaintext  M   M  E  S  A  J  D  E  P  R  O  B  A
    Key        K   C  H  E  I  A  M  E  A  C  H  E  I
    M (codes)     15  6 21  0 12  5  6 18 20 17  3  0
    K (codes)      4  9  6 10  0 15  6  0  4  9  6 10
    M+K (mod 31)  19 15 27 10 12 20 12 18 24 26  9 10
    Ciphertext C   Q  M  W  I  J  R  J  P  Ț  V  H  I

  Plaintext  (uppercase, no spaces) : MESAJDEPROBA
  Cryptogram                        : QMWIJRJPȚVHI
```

The three long messages are wrapped here to fit the page; on the terminal each is printed on one line.

The Tabula Recta

The second menu entry prints the square of section 1 for all 31 letters. Row `k`, column `m` holds the letter `(m + k) mod 31`, so the first row is the alphabet itself — the key letter `A` has code 0 and shifts nothing.

```
   A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z
A  A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z
Ă  Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z A
Â  Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z A Ă
B  B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â
C  C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B
D  D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C
E  E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D
F  F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E
G  G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F
H  H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F G
I  I Î J K L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F G H
Î  Î J K L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F G H I
J  J K L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F G H I Î
K  K L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F G H I Î J
L  L M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F G H I Î J K
M  M N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F G H I Î J K L
N  N O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F G H I Î J K L M
O  O P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F G H I Î J K L M N
P  P Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F G H I Î J K L M N O
Q  Q R S Ș T Ț U V W X Y Z A Ă Â B C D E F G H I Î J K L M N O P
R  R S Ș T Ț U V W X Y Z A Ă Â B C D E F G H I Î J K L M N O P Q
S  S Ș T Ț U V W X Y Z A Ă Â B C D E F G H I Î J K L M N O P Q R
Ș  Ș T Ț U V W X Y Z A Ă Â B C D E F G H I Î J K L M N O P Q R S
T  T Ț U V W X Y Z A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș
Ț  Ț U V W X Y Z A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T
U  U V W X Y Z A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț
V  V W X Y Z A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U
W  W X Y Z A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V
X  X Y Z A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W
Y  Y Z A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X
Z  Z A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y
```

Automated checks

The implementation is pinned to the example worked out by hand in the assignment. Because `encrypt` and `decrypt` take the ordered alphabet as a parameter, they can be run over the 26 letters of English and compared with the values printed in the laboratory work:

Input	Key	Expected	Alphabet
`PERASPERAADASTRA`	`SUPER`	`HYGEJHYGERVUHXIS`	English, n = 26
`HYGEJHYGERVUHXIS`	`SUPER`	`PERASPERAADASTRA`	English, n = 26

The individual columns of the table in the assignment are checked as well — `P(15) + S(18) = 7 → H` for the first and `A(0) + S(18) = 18 → S` for the last. The remaining tests cover the Romanian alphabet: that `decrypt(encrypt(m, k), k) = m` for a message containing diacritics and for the whole alphabet at once, that `c = (m + k) mod 31` holds for all 961 pairs of letters, that a key of all `A` leaves the text unchanged and a key of one letter degenerates into the Caesar cipher of Laboratory Work No. 1, that every row of the Tabula Recta is a permutation of the alphabet, and that the validation rejects short keys, foreign letters, digits and punctuation.

```
$ python3 -m unittest discover -s tests
..........................
----------------------------------------------------------------------
Ran 26 tests in 0.005s

OK
```

4. Conclusions

The Vigenère cipher fixes the one defect that destroyed the ciphers of the first two laboratory works, and it does so with almost no extra machinery: the code of this implementation differs from the Caesar cipher of Laboratory Work No. 1 in a single detail — the shift is read from a repeating key stream instead of being a constant. That one change is enough to break the link between a plaintext letter and a ciphertext letter. The runs in section 3 show it concretely: in a message of 36 letters, one plaintext `A` already produces six different ciphertext letters, and one ciphertext `I` covers three different plaintext letters. The frequency count that broke the cryptogram of Laboratory Work No. 2 returns nothing useful here.

The cipher is still not secure, and it is worth being precise about why, because the reason is no longer the one from the previous works. It is not the key space: with keys of 7 letters or more over 31 letters, exhaustive search is already out of the question. The weakness is the repetition of the key. The cryptogram is in truth `|key|` Caesar ciphertexts interleaved, so an attacker who recovers the key length — by the Kasiski examination on repeated groups, or by the index of coincidence — reduces the problem to `|key|` independent monoalphabetic ciphers and solves each of them with the method of Laboratory Work No. 2. The longer the key relative to the message, the fewer letters fall in each column and the harder that last step becomes, which is why the assignment sets a minimum length and why the limiting case — a random key as long as the message, used once — is the only unbreakable cipher there is.

On the implementation side the lesson of Laboratory Work No. 1 held again and paid off twice. Keeping the alphabet as explicit ordered data, with a letter's code being its position in that list, meant that the 31 Romanian letters needed no special treatment at all — a plain letter enciphering to `Ț` is simply what `(6 + 18) mod 31` gives. Keeping the alphabet as a *parameter* meant the cipher could be verified against the 26-letter English example printed in the assignment before being trusted on Romanian. The part that took the most code was again not the cipher but the validation of the input, and printing the intermediate codes in the same six-row layout the assignment uses for working the cipher by hand turned out to be the most useful thing in the program: every result in section 3 can be checked column by column against the Tabula Recta without running anything.

5. Source code

The full source code is in the repository github.com/alo42/CS.

To run the program:

```
git clone git@github.com:alo42/CS.git
cd CS/LAB3
python3 main.py
```

To run the automated checks:

```
python3 -m unittest discover -s tests
```

The program needs only Python 3 and the standard library.